VectorDB – Build Your Own AI

Project Summary & Interview Preparation Guide

Purpose: Simple project introduction, working explanation, and viva/interview questions with answers.

Project overview

A C++ project that stores vector data, searches for similar vectors, and connects document search with an AI question-answering flow using Ollama. The document question-answering feature follows the idea of Retrieval-Augmented Generation (RAG).

Prepared from the project details shared in this chat.

1. Project Introduction (Speak This in an Interview)

“My project is called **VectorDB – Build Your Own AI**. It is a vector database application developed using C++17. Its main purpose is to store vector representations of data and find similar items using vector similarity search. It also has a document-based question-answering feature. Documents are divided into smaller chunks, and Ollama is used to create embeddings for those chunks. When a user asks a question, the system creates an embedding for the question, searches for related document chunks, and sends the retrieved context to a language model. The model then generates an answer. This process is called Retrieval-Augmented Generation, or RAG. The project has a web interface for its Search, Documents, and Ask AI features. I built it to understand vector search, embeddings, backend APIs, and how AI can answer questions using stored documents.”

2. Project Summary in Simple Words

What problem does it solve? Keyword search looks for matching words. Vector search can find information with a similar meaning by comparing numerical representations called embeddings.

What is a vector? A vector is a list of numbers. An embedding model converts text into a vector so that similar text can be compared.

What is an embedding? An embedding is a numerical representation of text. Text with related meaning can have vectors that are close to each other.

What is the document flow? A document is split into smaller chunks. Each chunk is converted into an embedding and stored. A question is also converted into an embedding, and the system searches for relevant chunks.

What is RAG? RAG means Retrieval-Augmented Generation. The system retrieves useful information from documents and gives it to a language model to help it answer.

What does Ollama do? Ollama runs the embedding and language models used by the current local setup.

3. Main Technologies

- **C++17:** Main language for the backend and core logic.
- **cpp-httplib:** Used to create HTTP routes and handle web requests.
- **HNSW:** Graph-based indexing method used for approximate nearest-neighbour search.
- **Ollama:** Provides local access to embedding and text-generation models.
- **nomic-embed-text:** Embedding model named in the current project setup.
- **llama3.2:** Language model named in the current project setup.
- **HTML, CSS, JavaScript:** Used for the browser-based user interface.

4. How the Project Works

- **1. User input:** The user searches, adds documents, or asks a question in the web interface.
- **2. HTTP request:** The frontend sends a request to the C++ backend.
- **3. Embedding:** Ollama converts document text or a question into a vector.
- **4. Retrieval:** The query vector is compared with stored vectors to find relevant document chunks.
- **5. Prompt creation:** Retrieved chunks are added as context to the user's question.
- **6. AI response:** The language model generates an answer using the question and context.
- **7. Display:** The backend returns the result and the frontend shows it.

5. Interview Questions & Simple Answers

Q1. What is VectorDB?

A database system designed to store and search vector embeddings. It can find items that are similar in meaning, not only items with matching words.

Q2. Why did you build this project?

To understand vector databases and how they can be connected to AI applications for semantic search and document question answering.

Q3. What is a vector?

An ordered list of numbers. An embedding model creates this list from text.

Q4. What is an embedding model?

A model that converts text into numbers so a computer can compare the meaning or similarity of text.

Q5. What is semantic search?

Search based on meaning and context instead of only exact keyword matches.

Q6. What is cosine similarity?

A measure of how similar two vectors point in direction. A higher score generally means the vectors are more similar.

Q7. What is HNSW?

Hierarchical Navigable Small World. It is a graph-based method that helps find nearby vectors quickly.

Q8. Why use HNSW?

It can speed up similarity search on larger collections. It is approximate, so it trades some exactness for speed.

Q9. What is RAG?

Retrieval-Augmented Generation. It retrieves useful information first and gives it to a language model to generate an answer.

Q10. Why split documents into chunks?

Large documents may be too long to process as one piece. Chunks make embedding and retrieval more focused.

Q11. What happens when a user asks a question?

The question is embedded, relevant document chunks are retrieved, and those chunks are added to a prompt sent to the language model.

Q12. What is Ollama used for?

It provides a way to run embedding and language models locally and communicate with them through an API.

Q13. Embedding model vs language model?

An embedding model converts text into vectors. A language model generates text responses from prompts.

Q14. Why use the same embedding model for documents and questions?

Their vectors must be compatible and have matching dimensions so similarity comparisons are meaningful.

Q15. What is an API?

A defined way for software parts to communicate. Here, the frontend sends HTTP requests to the C++ backend.

Q16. What is cpp-httplib?

A C++ header-only HTTP library used to create a web server and handle HTTP requests.

Q17. What does the frontend do?

It provides the user interface for vector search, document operations, and asking questions.

Q18. What does the backend do?

It handles requests, vector operations, document processing, model calls, and returns results.

Q19. Keyword search vs vector search?

Keyword search looks for matching words. Vector search compares embeddings to find content with similar meaning.

Q20. What are the project's limitations?

Answer quality depends on the documents, chunking, embedding model, and language model. Local AI also needs system resources and Ollama must be available.

Q21. What would you improve next?

I would add persistent storage, authentication, support for more document formats, retrieval evaluation, better error handling, and cloud model support.

Q22. How would you test the project?

Test vector insertion and search, document processing, embedding generation, relevant retrieval, AI responses, invalid inputs, and Ollama-unavailable errors.

Q23. What happens if Ollama is unavailable?

Model-related actions such as embedding generation or answer generation can fail. The app should show a clear error.

Q24. Is RAG the same as fine-tuning?

No. RAG retrieves external information at question time. Fine-tuning updates a model through additional training.

6. Useful Commands

Download embedding model: `ollama pull nomic-embed-text`

Download language model: `ollama pull llama3.2`

Start Ollama: `ollama serve`

Compile on Windows (MinGW): `g++ -std=c++17 -O2 -pthread main.cpp -o vectordb.exe`

Run on Windows: `.\vectordb.exe`

Compile on Linux: `g++ -std=c++17 -O2 -pthread main.cpp -o vectordb`

Run on Linux: `./vectordb`

7. Quick Revision Before the Interview

- VectorDB stores and searches vector embeddings.
- Embeddings turn text into numerical vectors.
- Semantic search finds results by meaning.
- HNSW helps speed up nearest-neighbour search.
- RAG retrieves document context before AI generation.
- Ollama runs the models in the current local setup.
- The frontend communicates with the C++ backend through HTTP APIs.

Important accuracy note: This guide is based on the project details shared in the conversation. Before the interview, confirm the actual server port, routes, storage persistence, and implemented HNSW/document behaviour in your current source code. Do not claim a feature is complete unless it is present in your code.